

## NOTES

# Basic to Advance Queries and Joins



# Basic to Advance Queries and Joins

In this module, you will cover SQL querying concepts, from basic operators like AND, OR, and NOT, to advanced techniques, joins, and subqueries. The goal is to make the content digestible and practical using clear explanations.

## Basic to Advance Queries

### AND, OR, NOT Operators

**What:** These operators are used to filter data based on multiple conditions in MySQL queries. They allow the combination of multiple expressions, helping to refine the result set based on specific criteria.

- **AND:** Ensures that all conditions specified must be true for a record to be included in the result.
- **OR:** Includes records where at least one condition is true.
- **NOT:** Reverses the result of the condition, excluding records that meet the specified condition.

**Why:** They help in querying data with more precision by applying complex filtering conditions in a single query. This can be useful in an e-commerce scenario where you need to filter data based on customer behaviour, product selection, or order details.

#### AND Operator Example:

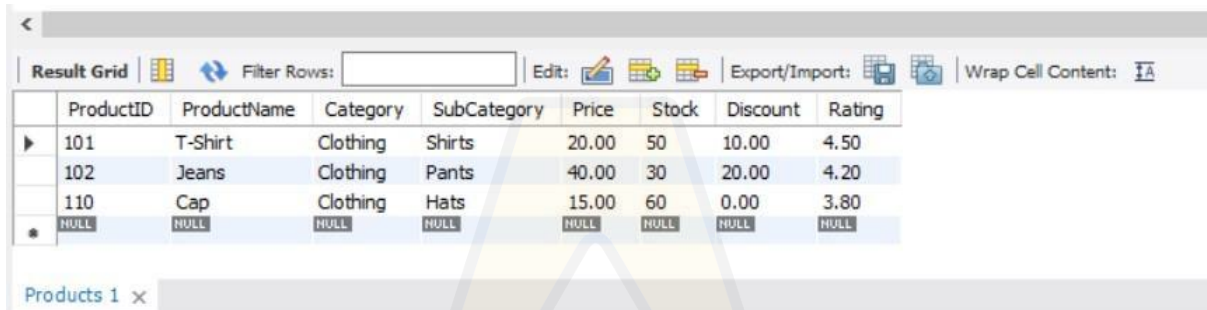
Retrieve orders where the quantity is greater than 5 **AND** the product category is "Shirts".

Unset

```
SELECT * FROM Products
WHERE Stock > 10 AND Category = 'Clothing';
```

This query retrieves all products from the Products table where the stock is greater than 10 and the product's category is "Clothing." This ensures that only products that meet both criteria are included in the result set.

### Output:



|   | ProductID | ProductName | Category | SubCategory | Price | Stock | Discount | Rating |
|---|-----------|-------------|----------|-------------|-------|-------|----------|--------|
| ▶ | 101       | T-Shirt     | Clothing | Shirts      | 20.00 | 50    | 10.00    | 4.50   |
|   | 102       | Jeans       | Clothing | Pants       | 40.00 | 30    | 20.00    | 4.20   |
|   | 110       | Cap         | Clothing | Hats        | 15.00 | 60    | 0.00     | 3.80   |
| * | NULL      | NULL        | NULL     | NULL        | NULL  | NULL  | NULL     | NULL   |

Products 1 x

### OR Operator Example:

Retrieve orders where the product is either "Shirts" **OR** the discount is greater than 20%.

Unset

```
SELECT * FROM Products
WHERE Category = 'Clothing' OR Discount > 20;
```

This query fetches all products from the Products table where the category is "Clothing" or the discount is greater than 20. This allows for a result set that includes products that meet either of the specified conditions.

## Output:

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
| 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
| 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
| 104       | Laptop      | Electronics | Computers   | 800.00 | 5     | 50.00    | 4.80   |
| 107       | Sweater     | Clothing    | Shirts      | 35.00  | 8     | 30.00    | 4.30   |
| 110       | Cap         | Clothing    | Hats        | 15.00  | 60    | 0.00     | 3.80   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

## NOT Operator Example:

Retrieve orders where the product is **NOT** in the "Electronics" category.

Unset

```
SELECT * FROM Products
WHERE NOT Category = 'Electronics';
```

This query returns all products where the Category is **not** "Electronics". It effectively excludes all records that belong to the "Electronics" category.

## Output:

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
| 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
| 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
| 105       | Bag         | Accessories | Bags        | 30.00  | 12    | 15.00    | 4.00   |
| 107       | Sweater     | Clothing    | Shirts      | 35.00  | 8     | 30.00    | 4.30   |
| 108       | Watch       | Accessories | Watches     | 150.00 | 25    | 0.00     | 4.10   |
| 110       | Cap         | Clothing    | Hats        | 15.00  | 60    | 0.00     | 3.80   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

# LIMIT

**What:** The LIMIT clause is used to specify the number of records to return from a query result set. It is especially useful when you want to restrict the output to a certain number of rows, which can be helpful in large datasets where you don't need to retrieve all data at once.

**Why:** Using LIMIT is essential when dealing with paginated results, previewing data, or showing top records (like top-selling products) from a large dataset. It helps optimise performance and ensures you get the desired subset of data quickly without overwhelming the system or user.

## Example: Retrieve the First 5 Orders

Unset

```
SELECT * FROM Products LIMIT 5;
```

This query retrieves the first 5 records from the Products table. It provides a quick view of a limited number of products without any specific ordering.

## Output:

35

36 • `SELECT * FROM Products LIMIT 5;`

Result Grid |   Filter Rows:  | Edit:    | Export/Import:  

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
| 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
| 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
| 104       | Laptop      | Electronics | Computers   | 800.00 | 5     | 50.00    | 4.80   |
| 105       | Bag         | Accessories | Bags        | 30.00  | 12    | 15.00    | 4.00   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

## Example: Retrieve Top 5 Highest Discount Products

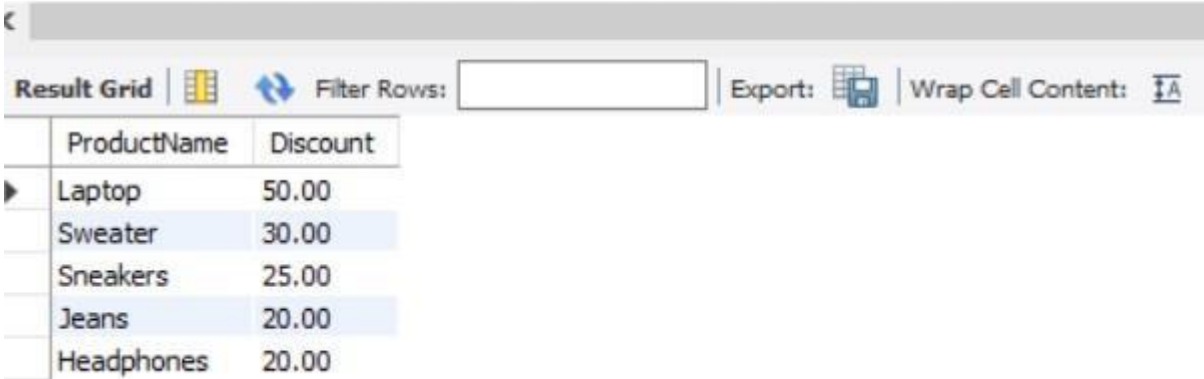
Unset

```
SELECT ProductName,Discount
FROM Products ORDER BY Discount DESC LIMIT 5;
```

This query selects the ProductName and Discount columns from the Products table, ordering the results by discount in descending order. It limits the output to the top 5 products with the highest discounts.

**Output:**

```
38
39 • SELECT ProductName, Discount
40 FROM Products ORDER BY Discount DESC LIMIT 5;
```



| ProductName | Discount |
|-------------|----------|
| Laptop      | 50.00    |
| Sweater     | 30.00    |
| Sneakers    | 25.00    |
| Jeans       | 20.00    |
| Headphones  | 20.00    |

**Example: Using LIMIT with OFFSET (Paginated Results)**

Unset

```
SELECT * FROM Products
ORDER BY ProductName ASC LIMIT 5 OFFSET 5;
```

This query retrieves records from the Products table, ordering them by ProductName in ascending order. It skips the first 5 records and then limits the result to the next 5 products, effectively returning products ranked 6 through 10 by name.



## Output:

```

41
42 • SELECT * FROM Products
43 ORDER BY ProductName ASC LIMIT 5 OFFSET 5;

```

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 106       | Smartphone  | Electronics | Mobiles     | 500.00 | 20    | 5.00     | 4.60   |
| 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
| 107       | Sweater     | Clothing    | Shirts      | 35.00  | 8     | 30.00    | 4.30   |
| 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
| 108       | Watch       | Accessories | Watches     | 150.00 | 25    | 0.00     | 4.10   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

# LIKE and Wildcard Characters

## Like

**What:** The LIKE operator in SQL is used to search for a specified pattern within a column. This operator is especially powerful when used in combination with wildcard characters, allowing you to perform flexible searches.

**Why:** The LIKE operator is essential for finding records that don't have exact matches but share a common pattern. This is often useful in scenarios like searching for products by partial names, filtering based on incomplete user input, or dealing with uncertain data.

## Wildcard Characters

Wildcard characters are special symbols that represent one or more characters in a string. The two primary wildcard characters used with the LIKE operator are:



- **%**: Matches any sequence of characters (zero or more).
- **\_**: Matches a single character.

## Example 1:

Find Products Whose Names Start with "Shirt"

Unset

```
SELECT * FROM Products
WHERE ProductName LIKE 'T%';
```

This query retrieves all products from the Products table where the product name starts with "Shirt" followed by any characters. The % wildcard allows for any characters to follow the word "Shirt", so results might include "Shirts", "Shirt Dress", etc.

### Output:

44

45 • `SELECT * FROM Products`

46 `WHERE ProductName LIKE 'T%';`

Result Grid |   Filter Rows:  | Edit:    | Export/Import:  

|   | ProductID | ProductName | Category | SubCategory | Price | Stock | Discount | Rating |
|---|-----------|-------------|----------|-------------|-------|-------|----------|--------|
| ▶ | 101       | T-Shirt     | Clothing | Shirts      | 20.00 | 50    | 10.00    | 4.50   |
| * | NULL      | NULL        | NULL     | NULL        | NULL  | NULL  | NULL     | NULL   |

## Example 2:

Find Products Containing the Word "Bag"

Unset

```
SELECT * FROM Products
WHERE ProductName LIKE '%Bag%';
```

This query looks for any products where the product name contains the word "Bag", regardless of where it appears in the product name. The % wildcard before and after "Bag" means that there can be any sequence of characters before and after the word "Bag". Results may include "Handbag", "Bagpack", "Shopping Bag", etc.

Output:

```

47
48 • SELECT * FROM Products
49 WHERE ProductName LIKE '%Bag%';
    
```

| ProductID | ProductName | Category    | SubCategory | Price | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|-------|-------|----------|--------|
| 105       | Bag         | Accessories | Bags        | 30.00 | 12    | 15.00    | 4.00   |
| NULL      | NULL        | NULL        | NULL        | NULL  | NULL  | NULL     | NULL   |

### Example 3:

Find Products Ending with "Shoes"

Unset

```

SELECT * FROM Products
WHERE ProductName LIKE '%Shoes';
    
```

This query retrieves products whose names end with the word "Shoes". The % wildcard before "Shoes" indicates that any sequence of characters may precede "Shoes". Results could include products like "Running Shoes", "Casual Shoes", or "Leather Shoes".

## Output:

```

50
51 • SELECT * FROM Products
52 WHERE ProductName LIKE '%Shoes';

```

| ProductID | ProductName | Category | SubCategory | Price | Stock | Discount | Rating |
|-----------|-------------|----------|-------------|-------|-------|----------|--------|
| NULL      | NULL        | NULL     | NULL        | NULL  | NULL  | NULL     | NULL   |

## Example 4:

Find Products with a Name that Has Exactly 5 Characters

Unset

```

SELECT * FROM Products
WHERE ProductName LIKE '_____';

```

This query retrieves products where the name is exactly 5 characters long. Each underscore (\_) represents a single character. So, products like "Shirt", "Pants", or "Shoes" (with exactly 5 characters) would be returned in the result set.

## Output:

```

53
54 • SELECT * FROM Products
55 WHERE ProductName LIKE '_____';

```

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
| 108       | Watch       | Accessories | Watches     | 150.00 | 25    | 0.00     | 4.10   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

# IN and BETWEEN Operators

## What:

- **IN:** The **IN** operator is used to filter records based on a list of specified values. It checks if a value matches any value in the provided list, making it useful for querying multiple values without needing multiple **OR** conditions.
- **BETWEEN:** The **BETWEEN** operator is used to retrieve records within a specified range, including the endpoints. This is particularly useful for filtering numerical, date, or time data where you want to find values that fall between two limits.

**Why:** Using **IN** and **BETWEEN** helps streamline your queries and improve readability, allowing you to filter data based on multiple specific values or ranges effectively. This is especially useful in e-commerce scenarios, where you may want to analyse sales data, product information, or order quantities.

## Example 1:

Find Orders Where Quantity Is Between 10 and 20

Unset

```
SELECT * FROM Products  
WHERE Stock BETWEEN 10 AND 50;
```

This query retrieves all products from the Products table where the stock quantity is between 10 and 50, inclusive. It filters the results to include only those products that fall within this specified range.

## Output:

56

57 • `SELECT * FROM Products`

58 `WHERE Stock BETWEEN 10 AND 50;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: 

|   | ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|---|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| ▶ | 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
|   | 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
|   | 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
|   | 105       | Bag         | Accessories | Bags        | 30.00  | 12    | 15.00    | 4.00   |
|   | 106       | Smartphone  | Electronics | Mobiles     | 500.00 | 20    | 5.00     | 4.60   |
|   | 108       | Watch       | Accessories | Watches     | 150.00 | 25    | 0.00     | 4.10   |
|   | 109       | Headphones  | Electronics | Audio       | 100.00 | 40    | 20.00    | 4.40   |
| • | NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

Products 12 x

## Example 2:

### Find Orders for Specific Product IDs

Unset

```
SELECT * FROM Products
WHERE ProductID IN (101, 102, 103);
```

This query selects all products from the Products table where the ProductID matches any of the specified values (101, 102, or 103). It provides a way to filter results based on a specific list of product IDs.

## Output:

59

60 • `SELECT * FROM Products`

61 `WHERE ProductID IN (101, 102, 103);`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: 

|   | ProductID | ProductName | Category | SubCategory | Price | Stock | Discount | Rating |
|---|-----------|-------------|----------|-------------|-------|-------|----------|--------|
| ▶ | 101       | T-Shirt     | Clothing | Shirts      | 20.00 | 50    | 10.00    | 4.50   |
|   | 102       | Jeans       | Clothing | Pants       | 40.00 | 30    | 20.00    | 4.20   |
|   | 103       | Sneakers    | Footwear | Shoes       | 60.00 | 15    | 25.00    | 4.70   |
| • | NULL      | NULL        | NULL     | NULL        | NULL  | NULL  | NULL     | NULL   |

## Example 3:

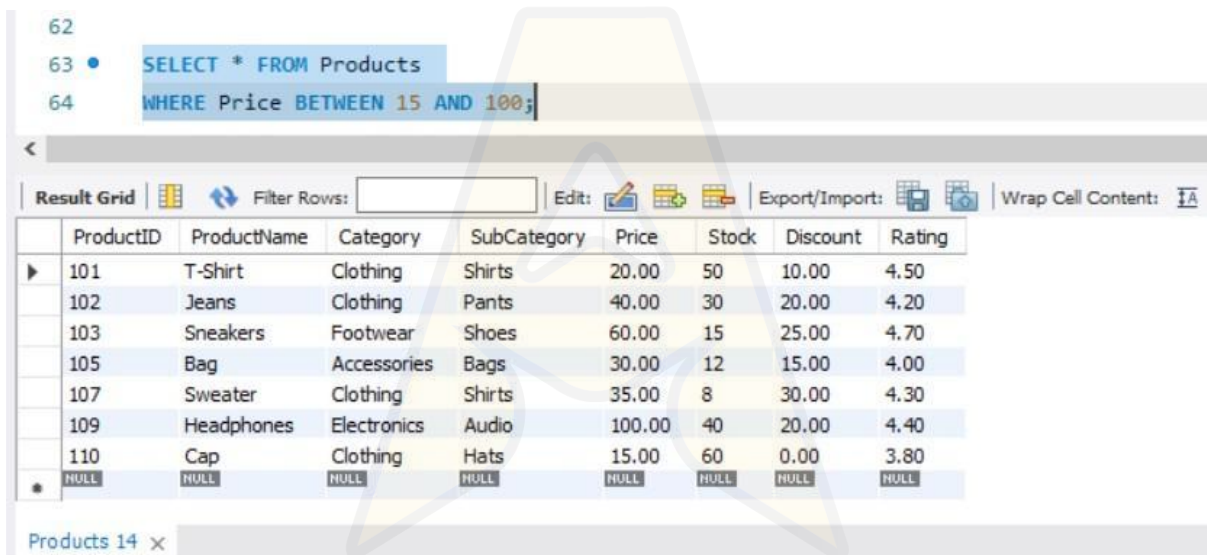
Find Products Priced Between \$50 and \$100

Unset

```
SELECT * FROM Products
WHERE Price BETWEEN 15 AND 100;
```

This query fetches all products from the Products table where the price is between 15 and 100, inclusive. It limits the results to products that fall within this price range.

### Output:



The screenshot shows a SQL query editor with the following query entered:

```
SELECT * FROM Products
WHERE Price BETWEEN 15 AND 100;
```

Below the query editor, a "Result Grid" displays the query results. The table has 9 columns: ProductID, ProductName, Category, SubCategory, Price, Stock, Discount, and Rating. The results show 14 rows of data, including products like T-Shirt, Jeans, Sneakers, Bag, Sweater, Headphones, and Cap, all with prices within the specified range. The last row shows NULL values for all columns.

| ProductID | ProductName | Category    | SubCategory | Price  | Stock | Discount | Rating |
|-----------|-------------|-------------|-------------|--------|-------|----------|--------|
| 101       | T-Shirt     | Clothing    | Shirts      | 20.00  | 50    | 10.00    | 4.50   |
| 102       | Jeans       | Clothing    | Pants       | 40.00  | 30    | 20.00    | 4.20   |
| 103       | Sneakers    | Footwear    | Shoes       | 60.00  | 15    | 25.00    | 4.70   |
| 105       | Bag         | Accessories | Bags        | 30.00  | 12    | 15.00    | 4.00   |
| 107       | Sweater     | Clothing    | Shirts      | 35.00  | 8     | 30.00    | 4.30   |
| 109       | Headphones  | Electronics | Audio       | 100.00 | 40    | 20.00    | 4.40   |
| 110       | Cap         | Clothing    | Hats        | 15.00  | 60    | 0.00     | 3.80   |
| NULL      | NULL        | NULL        | NULL        | NULL   | NULL  | NULL     | NULL   |

Products 14 x

## Aliases

**What:** Aliases are temporary names assigned to tables or columns in a SQL query. They are used to improve the readability and clarity of the query, especially when dealing with complex expressions, long table names, or when you want to give a more meaningful name to the output columns.

**Why:** Using aliases simplifies the presentation of query results and makes SQL statements easier to understand. This is particularly

helpful when creating reports or dashboards, allowing stakeholders to quickly grasp the data's meaning without getting bogged down by technical column or table names.

## Example 1:

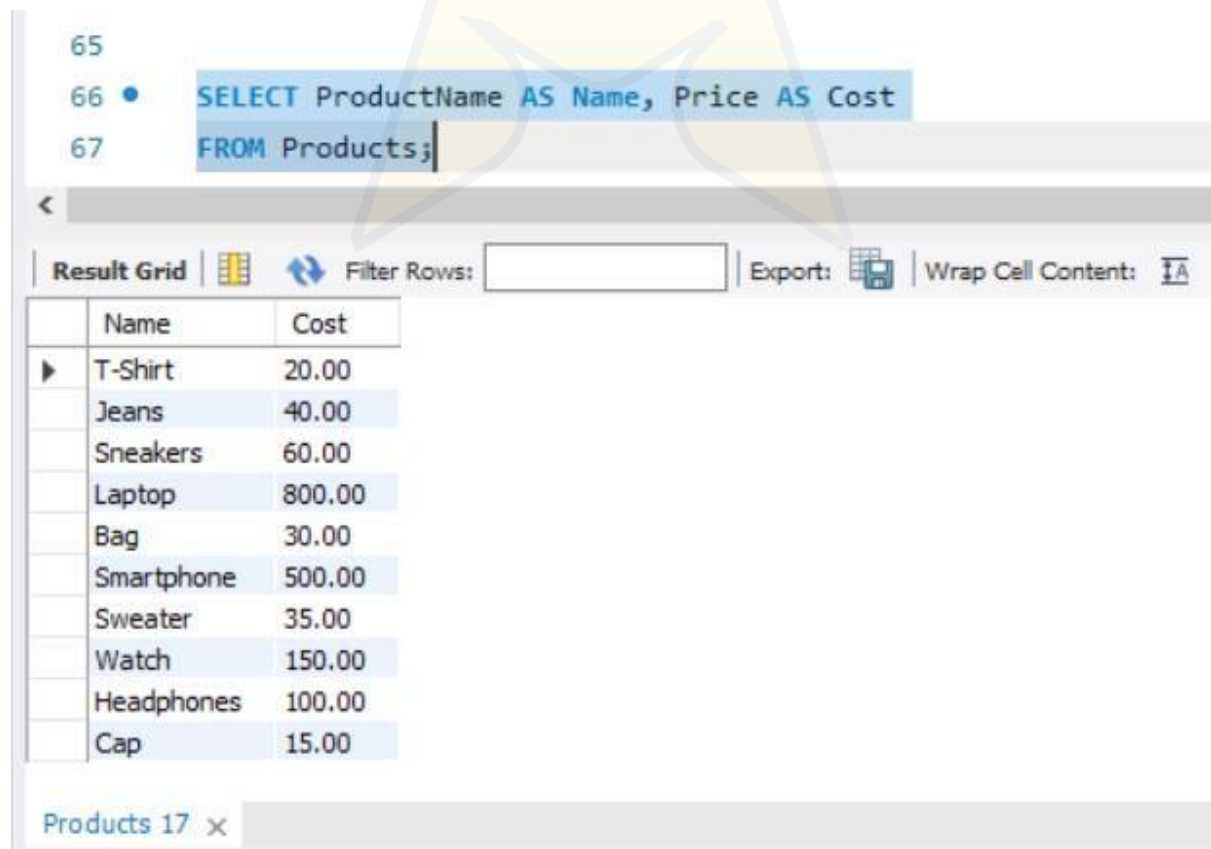
### Simplifying Column Names

Unset

```
SELECT ProductName AS Name, Price AS Cost
FROM Products;
```

This query retrieves the product names and prices from the Products table but assigns the aliases "Name" and "Cost" to the columns. This makes the output clearer and more user-friendly, especially when presenting data to non-technical audiences.

### Output:



The screenshot shows a SQL query editor with the following query:

```
SELECT ProductName AS Name, Price AS Cost
FROM Products;
```

Below the query editor, the result grid displays the output of the query. The grid has two columns: Name and Cost. The data is as follows:

|   | Name       | Cost   |
|---|------------|--------|
| ▶ | T-Shirt    | 20.00  |
|   | Jeans      | 40.00  |
|   | Sneakers   | 60.00  |
|   | Laptop     | 800.00 |
|   | Bag        | 30.00  |
|   | Smartphone | 500.00 |
|   | Sweater    | 35.00  |
|   | Watch      | 150.00 |
|   | Headphones | 100.00 |
|   | Cap        | 15.00  |

At the bottom of the result grid, there is a tab labeled "Products 17" with a close button (x).



## Example 2:

### Alias for Calculated Columns

Unset

```
SELECT ProductName, Price, (Price * Stock) AS TotalCost FROM
Products;
```

Here, we calculate the total cost of each product in the OrderDetails table by multiplying the Price by Stock. The result is given an alias "TotalCost," making it clear what this column represents in the output. This is especially useful in financial reporting where calculated values need clear labels.

### Output:

65

66 • SELECT ProductName, Price, (Price \* Stock) AS TotalCost |

67 FROM Products;

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | ProductName | Price  | TotalCost |
|---|-------------|--------|-----------|
| ▶ | T-Shirt     | 20.00  | 1000.00   |
|   | Jeans       | 40.00  | 1200.00   |
|   | Sneakers    | 60.00  | 900.00    |
|   | Laptop      | 800.00 | 4000.00   |
|   | Bag         | 30.00  | 360.00    |
|   | Smartphone  | 500.00 | 10000.00  |
|   | Sweater     | 35.00  | 280.00    |
|   | Watch       | 150.00 | 3750.00   |
|   | Headphones  | 100.00 | 4000.00   |
|   | Cap         | 15.00  | 900.00    |

Result 15 x

# GROUP BY and HAVING Clauses

## What is GROUP BY?

The **GROUP BY** clause in SQL is used to group rows that have the same values in specified columns into summary rows. This is commonly used with aggregate functions like **COUNT()**, **SUM()**, **AVG()**, etc., to perform calculations on each group of data.

## GROUP BY?

The **GROUP BY** clause is essential for summarising data by categories. For example, if you want to know how many products are in each category or calculate total sales by category, **GROUP BY** provides an efficient way to organise the data accordingly.

## Example of GROUP BY

Unset

```
SELECT Category, COUNT(*) AS ProductCount  
FROM Products  
GROUP BY Category;
```

This query counts the number of products in each category. It groups all rows in the Products table by the Category column and counts how many products exist in each category, returning a summary result set.

**Output:**

```
74
75 • SELECT Category, COUNT(*) AS ProductCount
76 FROM Products
77 GROUP BY Category;
```

| Category    | ProductCount |
|-------------|--------------|
| Clothing    | 4            |
| Footwear    | 1            |
| Electronics | 3            |
| Accessories | 2            |

## What is HAVING?

The **HAVING** clause is used in conjunction with the **GROUP BY** clause. It works similarly to the **WHERE** clause but is specifically designed to filter records after the aggregation has occurred. This allows you to apply conditions to aggregate values.

## Why HAVING?

**HAVING** is useful for filtering groups based on the results of aggregate functions, enabling you to refine the output to only include groups that meet specific criteria.

## Example of HAVING

Unset

```
SELECT Category, AVG(Price) AS AvgPrice
FROM Products
GROUP BY Category
HAVING AVG(Price) > 30;
```

This query retrieves categories where the average price of products is greater than ₹30. It first groups the products by category, calculates the average price for each category, and then filters the results to include only those categories with an average price exceeding ₹30.

### Output:

```

78
79 • SELECT Category, AVG(Price) AS AvgPrice
80 FROM Products
81 GROUP BY Category
82 HAVING AVG(Price) > 30;

```

| Category    | AvgPrice   |
|-------------|------------|
| Footwear    | 60.000000  |
| Electronics | 466.666667 |
| Accessories | 90.000000  |

## Example Using GROUP BY and HAVING

Unset

```

SELECT Category, COUNT(*) AS DiscountedProductCount
FROM Products
GROUP BY Category
HAVING COUNT(*) > 2 AND AVG(Discount) > 15;

```

This query counts the number of products in each category from the Products table, focusing specifically on those categories that have more than 2 products and an average discount greater than 15. The results are grouped by Category, with the count labelled as DiscountedProductCount.

**Output:**

```
83
84 • SELECT Category, COUNT(*) AS DiscountedProductCount
85 FROM Products
86 GROUP BY Category
87 HAVING COUNT(*) > 2 AND AVG(Discount) > 15;
```

<

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | Category    | DiscountedProductCount |
|---|-------------|------------------------|
| ▶ | Electronics | 3                      |

## CASE Statement

### What is a CASE Statement?

The **CASE** statement in SQL is a powerful tool that allows you to incorporate conditional logic directly into your queries. It functions similarly to **IF-THEN-ELSE** structures found in many programming languages, enabling you to evaluate different conditions and return specific values based on those evaluations. This capability is essential when you want to create more readable and meaningful output from your queries, as it allows you to dynamically categorise or modify data based on specific criteria.

### Why Use CASE?

Using a **CASE** statement can significantly enhance the utility of your SQL queries by:

- **Categorizing Data:** You can group data into different categories based on specified conditions, making analysis easier and more intuitive.

- **Creating Derived Columns:** It allows the addition of new columns to your result set without altering the underlying data, providing insights at a glance.
- **Enhancing Readability:** By using descriptive labels in your results, you make your output easier to interpret for stakeholders or team members.

## Example of a CASE Statement

### Categorizing Products by Stock Status

```
Unset
SELECT ProductID, ProductName, Stock,
       CASE
           WHEN Stock > 20 THEN 'In Stock'
           WHEN Stock BETWEEN 1 AND 20 THEN 'Low Stock'
           ELSE 'Out of Stock'
       END AS StockStatus
FROM Products;
```

This query retrieves the ProductID, ProductName, and Stock from the Products table. It uses a CASE statement to categorise the stock status as 'In Stock', 'Low Stock', or 'Out of Stock' based on the quantity in stock. The resulting stock status is labelled as StockStatus.

## Output:

```

88
89 • SELECT ProductID, ProductName, Stock,
90     CASE
91         WHEN Stock > 20 THEN 'In Stock'
92         WHEN Stock BETWEEN 1 AND 20 THEN 'Low Stock'
93         ELSE 'Out of Stock'
94     END AS StockStatus
95 FROM Products;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | ProductID | ProductName | Stock | StockStatus |
|---|-----------|-------------|-------|-------------|
| ▶ | 101       | T-Shirt     | 50    | In Stock    |
|   | 102       | Jeans       | 30    | In Stock    |
|   | 103       | Sneakers    | 15    | Low Stock   |
|   | 104       | Laptop      | 5     | Low Stock   |
|   | 105       | Bag         | 12    | Low Stock   |
|   | 106       | Smartphone  | 20    | Low Stock   |
|   | 107       | Sweater     | 8     | Low Stock   |
|   | 108       | Watch       | 25    | In Stock    |
|   | 109       | Headphones  | 40    | In Stock    |
|   | 110       | Cap         | 60    | In Stock    |

Result 22 x

## Product Rating Categories

Unset

```

SELECT ProductID, ProductName, Rating,
       CASE
           WHEN Rating >= 4.5 THEN 'Excellent'
           WHEN Rating >= 3 THEN 'Good'
           ELSE 'Needs Improvement'
       END AS RatingCategory
FROM Products;

```

This query selects the ProductID, ProductName, and Rating from the Products table. It employs a CASE statement to classify the rating



into categories: 'Excellent', 'Good', or 'Needs Improvement' based on the rating value. The resulting category is labelled as RatingCategory.

### Output:

```

96
97 • SELECT ProductID, ProductName, Rating,
98     CASE
99         WHEN Rating >= 4.5 THEN 'Excellent'
100        WHEN Rating >= 3 THEN 'Good'
101        ELSE 'Needs Improvement'
102     END AS RatingCategory
103 FROM Products;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

|   | ProductID | ProductName | Rating | RatingCategory |
|---|-----------|-------------|--------|----------------|
| ▶ | 101       | T-Shirt     | 4.50   | Excellent      |
|   | 102       | Jeans       | 4.20   | Good           |
|   | 103       | Sneakers    | 4.70   | Excellent      |
|   | 104       | Laptop      | 4.80   | Excellent      |
|   | 105       | Bag         | 4.00   | Good           |
|   | 106       | Smartphone  | 4.60   | Excellent      |
|   | 107       | Sweater     | 4.30   | Good           |
|   | 108       | Watch       | 4.10   | Good           |
|   | 109       | Headphones  | 4.40   | Good           |
|   | 110       | Cap         | 3.80   | Good           |

Result 23 x

## Discount Level Classification

Unset

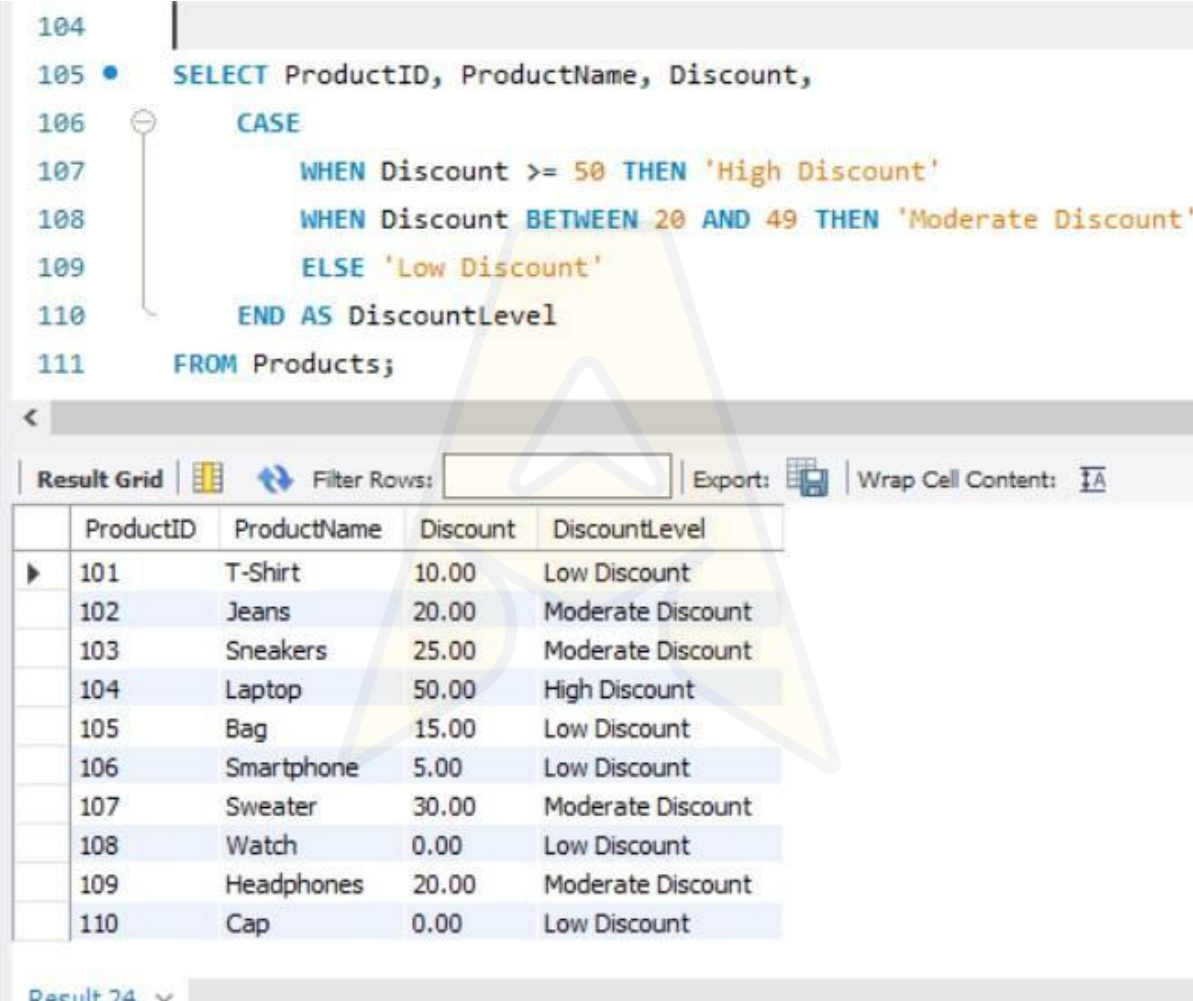
```

SELECT ProductID, ProductName, Discount,
       CASE
           WHEN Discount >= 50 THEN 'High Discount'
           WHEN Discount BETWEEN 20 AND 49 THEN 'Moderate Discount'
           ELSE 'Low Discount'
       END AS DiscountLevel
FROM Products;

```

This query retrieves the ProductID, ProductName, and Discount from the Products table. It uses a CASE statement to categorise discounts as 'High Discount', 'Moderate Discount', or 'Low Discount' based on the discount percentage. The resulting classification is labelled as DiscountLevel.

### Output:



```

104
105 • SELECT ProductID, ProductName, Discount,
106     CASE
107         WHEN Discount >= 50 THEN 'High Discount'
108         WHEN Discount BETWEEN 20 AND 49 THEN 'Moderate Discount'
109         ELSE 'Low Discount'
110     END AS DiscountLevel
111 FROM Products;
  
```

Result Grid

|   | ProductID | ProductName | Discount | DiscountLevel     |
|---|-----------|-------------|----------|-------------------|
| ▶ | 101       | T-Shirt     | 10.00    | Low Discount      |
|   | 102       | Jeans       | 20.00    | Moderate Discount |
|   | 103       | Sneakers    | 25.00    | Moderate Discount |
|   | 104       | Laptop      | 50.00    | High Discount     |
|   | 105       | Bag         | 15.00    | Low Discount      |
|   | 106       | Smartphone  | 5.00     | Low Discount      |
|   | 107       | Sweater     | 30.00    | Moderate Discount |
|   | 108       | Watch       | 0.00     | Low Discount      |
|   | 109       | Headphones  | 20.00    | Moderate Discount |
|   | 110       | Cap         | 0.00     | Low Discount      |

Result 24 x

# Joins in MySQL

## What are Joins?

A **JOIN** is a SQL operation used to combine rows from two or more tables based on a related column. Joins help retrieve related data stored in multiple tables, allowing you to query and manipulate the data as a unified entity.

## Why Use Joins?

Joins enable you to combine data from multiple tables into a single result set. For example, you can combine product and order details to analyse the relationships between them.

## Types of Joins

**INNER JOIN:** Retrieve data from both tables based on matching keys.

**LEFT JOIN:** Get all records from left table (Priority table) and additional features from another table based on common key, else generate null for key not found.

**RIGHT JOIN:** Get all records from right table (Priority table) and additional features from another table based on common key, else generate null for key not found.

**FULL JOIN:** Get all possible records / available keys from both tables. Where key found the corresponding data will be added else generates null.

**CROSS JOIN:** Generate Combinations of Rows from Two Tables

# INNER JOIN

## What is an INNER JOIN?

An **INNER JOIN** in SQL is used to combine rows from two or more tables based on a related column between them. The result of an **INNER JOIN** includes only those records where there is a match in both tables. If no match is found, those rows are excluded from the result set.

## Why Use INNER JOIN?

The **INNER JOIN** is used when you need to retrieve data that exists in both tables and is linked by a common column, such as a `Product_id`. This is ideal when you want to focus only on the intersection of data between tables and exclude records that don't match.

- **Data Integration:** Combine information from related tables to form comprehensive datasets.
- **Relational Database:** Helps in querying and linking data stored across multiple related tables.
- **Efficiency:** Filters out unrelated data to focus on the relevant records that match across both tables.

## Table Creation

### Customers Table:

Unset

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    Country VARCHAR(50)  
);
```

## Orders Table:

Unset

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    Amount DECIMAL(10, 2),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

## Inserting Data:

Unset

```
-- Inserting data into Customers table  
INSERT INTO Customers (CustomerID, CustomerName, Country)  
VALUES  
(1, 'Alice', 'USA'),  
(2, 'Bob', 'Canada'),  
(3, 'Charlie', 'UK'),  
(4, 'David', 'USA'),  
(5, 'Eve', 'Australia');  
  
-- Inserting data into Orders table  
INSERT INTO Orders (OrderID, CustomerID, OrderDate, Amount)  
VALUES  
(101, 1, '2024-01-10', 500.00),  
(102, 2, '2024-01-15', 300.00),  
(103, 1, '2024-01-20', 150.00),  
(104, 4, '2024-01-25', 700.00),  
(105, 3, '2024-01-30', 450.00),  
(106, NULL, '2024-02-05', 200.00); -- An order without a customer
```

## Example 1:

Unset

```
SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount  
FROM Customers  
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

This query retrieves the CustomerName, OrderID, and Amount from the Customers and Orders tables. It uses an INNER JOIN to combine records where the CustomerID matches in both tables, ensuring that only customers with corresponding orders are included in the results.

### Output:

```

148
149 • SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount
150 FROM Customers
151 INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
152

```

|   | CustomerName | OrderID | Amount |
|---|--------------|---------|--------|
| ▶ | Alice        | 101     | 500.00 |
|   | Alice        | 103     | 150.00 |
|   | Bob          | 102     | 300.00 |
|   | Charlie      | 105     | 450.00 |
|   | David        | 104     | 700.00 |

### Example 2:

Unset

```

SELECT Customers.CustomerName, Orders.OrderDate
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE Orders.Amount > 400;

```

This query selects the CustomerName and OrderDate from the Customers and Orders tables. It employs an INNER JOIN to link records based on matching CustomerIDs and filters the results to include only those orders where the Amount exceeds 400. This

ensures that only relevant orders from customers with significant purchases are displayed.

### Output:

```
152
153 • SELECT Customers.CustomerName, Orders.OrderDate
154 FROM Customers
155 INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID
156 WHERE Orders.Amount > 400;
157
```



The screenshot shows a database query result grid. The grid has two columns: 'CustomerName' and 'OrderDate'. It contains three rows of data: Alice (2024-01-10), David (2024-01-25), and Charlie (2024-01-30). The row for David is highlighted. Above the grid, there is a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'.

|   | CustomerName | OrderDate  |
|---|--------------|------------|
| ▶ | Alice        | 2024-01-10 |
|   | David        | 2024-01-25 |
|   | Charlie      | 2024-01-30 |

## LEFT JOIN (or LEFT OUTER JOIN)

### What is a LEFT JOIN?

A **LEFT JOIN** returns all the rows from the **left table** and the matched rows from the **right table**. If no match is found in the right table, the result is NULL for the right table's columns.

### Why Use LEFT JOIN?

The **LEFT JOIN** is useful when you want to ensure that you retrieve all rows from one table, regardless of whether they have matching records in the related table. For example, it can be used to find all products, including those that have not yet been ordered.



## Example 1:

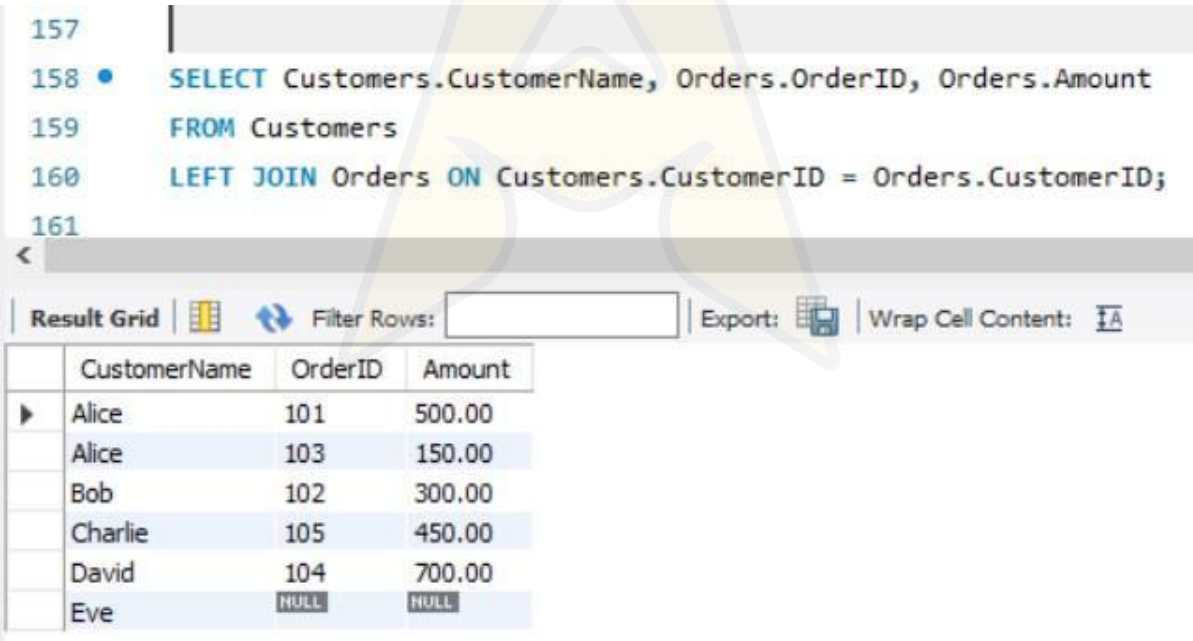
### Retrieve All Products and Their Reviews

Unset

```
SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

This query retrieves the CustomerName, OrderID, and Amount from the Customers and Orders tables. It uses a LEFT JOIN to include all customers, even those who do not have corresponding orders. For customers without orders, the OrderID and Amount will be returned as NULL.

### Output:



The screenshot shows a SQL query editor with the following query:

```
157
158 • SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount
159 FROM Customers
160 LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
161
```

Below the query, there is a "Result Grid" section with a table showing the results. The table has four columns: CustomerName, OrderID, and Amount. The results are as follows:

|   | CustomerName | OrderID | Amount |
|---|--------------|---------|--------|
| ▶ | Alice        | 101     | 500.00 |
|   | Alice        | 103     | 150.00 |
|   | Bob          | 102     | 300.00 |
|   | Charlie      | 105     | 450.00 |
|   | David        | 104     | 700.00 |
|   | Eve          | NULL    | NULL   |

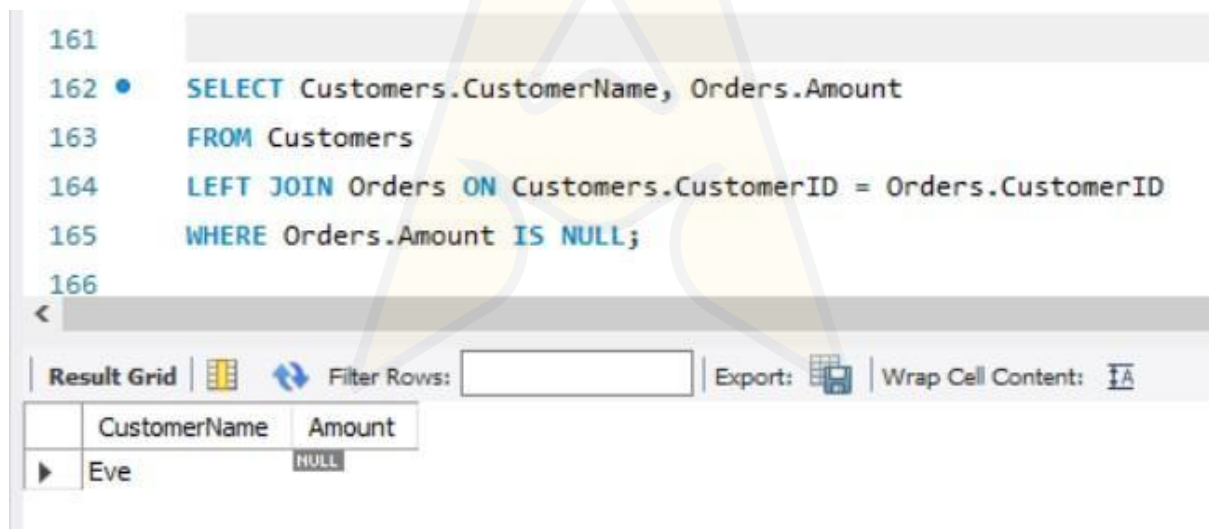
## Example 2:

Unset

```
SELECT Customers.CustomerName, Orders.Amount  
FROM Customers  
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID  
WHERE Orders.Amount IS NULL;
```

This query selects the CustomerName and Amount from the Customers and Orders tables. It employs a LEFT JOIN to include all customers and filters the results to show only those customers who do not have any corresponding orders (i.e., where the Amount is NULL). This effectively identifies customers with no orders placed.

### Output:



```
161  
162 • SELECT Customers.CustomerName, Orders.Amount  
163 FROM Customers  
164 LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID  
165 WHERE Orders.Amount IS NULL;  
166
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: IA

|   | CustomerName | Amount |
|---|--------------|--------|
| ▶ | Eve          | NULL   |

## RIGHT JOIN (or RIGHT OUTER JOIN)

### What is a RIGHT JOIN?

A **RIGHT JOIN** returns all rows from the **right table** and the matched rows from the **left table**. If no match is found in the left table, NULL values are returned for the left table's columns.

## Why Use RIGHT JOIN?

Use the **RIGHT JOIN** when you want to retrieve all records from the right table, even if there are no corresponding rows in the left table. For example, you may want to see all orders, including those that may not have associated products.

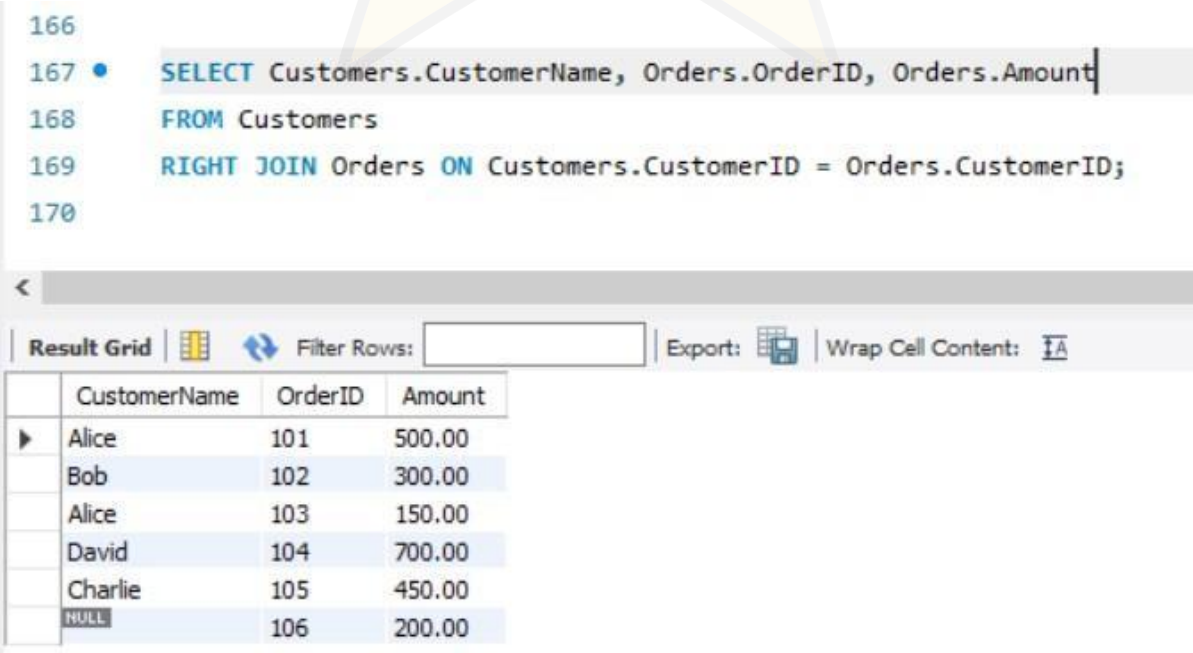
### Example 1:

Unset

```
SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

This query retrieves the CustomerName, OrderID, and Amount from the Customers and Orders tables. It uses a RIGHT JOIN to include all orders, even those that do not have corresponding customer records. For orders without associated customers, the CustomerName will be returned as NULL.

### Output:



```
166
167 • SELECT Customers.CustomerName, Orders.OrderID, Orders.Amount
168 FROM Customers
169 RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
170
```

|   | CustomerName | OrderID | Amount |
|---|--------------|---------|--------|
| ▶ | Alice        | 101     | 500.00 |
|   | Bob          | 102     | 300.00 |
|   | Alice        | 103     | 150.00 |
|   | David        | 104     | 700.00 |
|   | Charlie      | 105     | 450.00 |
|   | NULL         | 106     | 200.00 |

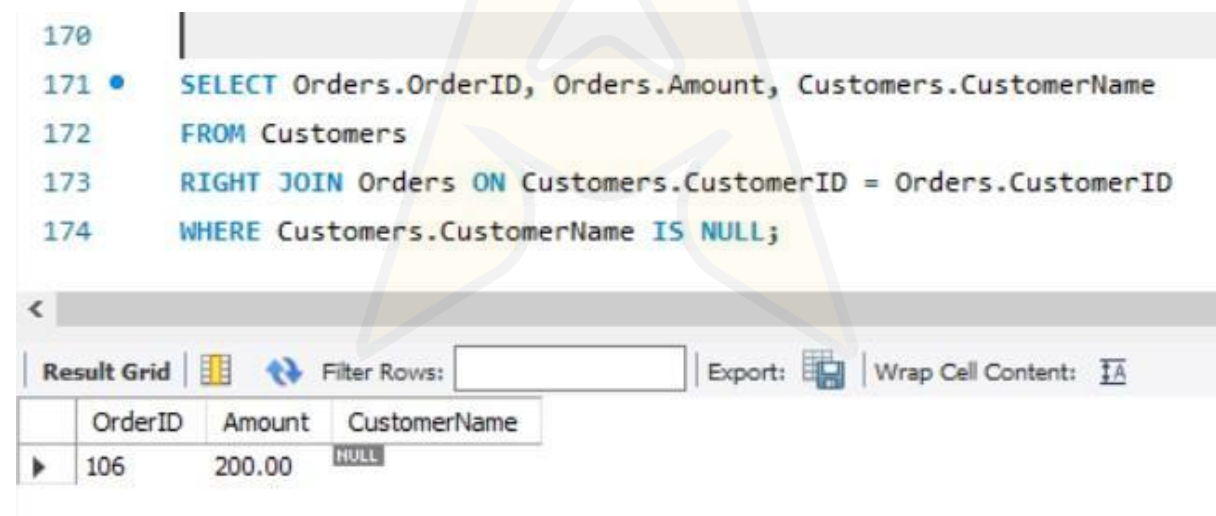
## Example 2:

Unset

```
SELECT Orders.OrderID, Orders.Amount, Customers.CustomerName
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE Customers.CustomerName IS NULL;
```

This query selects the OrderID, Amount, and CustomerName from the Orders and Customers tables. It employs a RIGHT JOIN to include all orders and filters the results to show only those orders that do not have any associated customers (i.e., where the CustomerName is NULL). This effectively identifies orders that were placed without a corresponding customer record.

### Output:



The screenshot shows a SQL query editor with the following query:

```
SELECT Orders.OrderID, Orders.Amount, Customers.CustomerName
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID
WHERE Customers.CustomerName IS NULL;
```

Below the query, the result grid displays the output:

| OrderID | Amount | CustomerName |
|---------|--------|--------------|
| 106     | 200.00 | NULL         |

## Subqueries

### What are Subqueries?

**Subqueries** are queries nested inside another SQL query. They are often used to retrieve intermediate data that can be utilised by the

main query. Subqueries can be placed in various clauses, such as **SELECT**, **FROM**, **WHERE**, and **HAVING**.

## Why Use Subqueries?

Subqueries help handle complex logic by breaking it down into smaller, manageable queries. They are useful for filtering data, performing comparisons, calculating intermediate results, and dynamically retrieving values used in the main query.

## Types of Subqueries in SQL

### Single-row Subqueries:

- **What:** A single-row subquery returns only one row of data, and it is typically used with comparison operators like **=**, **<**, **>**.
- **When to Use:** When you want to compare a single value in the main query.
- **Example:** Find the **CustomerName** of the customer with the highest **CustomerID**.

Unset

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    Country VARCHAR(50)  
);  
  
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    Amount DECIMAL(10, 2),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);  
  
-- Inserting data into Customers table  
INSERT INTO Customers (CustomerID, CustomerName, Country)
```

```
VALUES
(1, 'Alice', 'USA'),
(2, 'Bob', 'Canada'),
(3, 'Charlie', 'UK'),
(4, 'David', 'USA'),
(5, 'Eve', 'Australia');

-- Inserting data into Orders table
INSERT INTO Orders (OrderID, CustomerID, OrderDate, Amount)
VALUES
(101, 1, '2024-01-10', 500.00),
(102, 2, '2024-01-15', 300.00),
(103, 1, '2024-01-20', 150.00),
(104, 4, '2024-01-25', 700.00),
(105, 3, '2024-01-30', 450.00),
(106, NULL, '2024-02-05', 200.00); -- An order without a customer

SELECT CustomerName
FROM Customers
WHERE CustomerID = (SELECT MAX(CustomerID) FROM Customers);
```

This subquery retrieves a single value, which is then used in the outer query. The example finds the CustomerName of the customer with the highest CustomerID.

The inner query (SELECT MAX(CustomerID) FROM Customers) returns the maximum CustomerID, which is a single value. The outer query then selects the CustomerName corresponding to that CustomerID.

### Output:

| Result Grid |              | Filter Rows: |
|-------------|--------------|--------------|
|             | CustomerName |              |
| ▶           | Eve          |              |

## Multi-row Subqueries:

- **What:** Multi-row subqueries return multiple rows and are used with operators like IN, ANY, ALL that can handle multiple values.
- **When to Use:** When you need to compare or match against multiple rows.
- **Example:** Find all customers who have placed an order with an amount greater than 400.

Unset

```
SELECT CustomerName  
FROM Customers  
WHERE CustomerID IN (SELECT CustomerID FROM Orders WHERE Amount >  
400);
```

This subquery returns multiple rows and is typically used with operators like **IN**. The example finds all customers who have placed an order with an amount greater than 400.

The inner query retrieves a list of **CustomerIDs** from the **Orders** table where the order amount exceeds 400. The outer query then selects customer names whose IDs are in that list.

### Output:

| Result Grid |              | Filter Rows: |
|-------------|--------------|--------------|
|             | CustomerName |              |
| ▶           | Alice        |              |
|             | Charlie      |              |
|             | David        |              |



## Multiple Column Subqueries:

**What:** These return more than one column, often used for matching multiple attributes.

**When to Use:** When you want to retrieve or compare multiple fields from a subquery.

**Example:** Find all orders where the **OrderDate** is later than the order date of order ID 101, and return both **OrderID** and **Amount**.

Unset

```
SELECT OrderID, Amount
FROM Orders
WHERE (OrderDate, Amount) > (SELECT OrderDate, Amount FROM Orders
WHERE OrderID = 101);
```

This type of subquery returns multiple columns. The example retrieves orders where the OrderDate is later than that of order ID 101, returning both OrderID and Amount.

The inner subquery selects both the OrderDate and Amount for order ID 101. The outer query compares each order's date and amount against these values to find matching records.

**Output:**

| Result Grid   Filter Rows: |         |        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------|
|                                                                                                                                                                                                  | OrderID | Amount |
| ▶                                                                                                                                                                                                | 102     | 300.00 |
|                                                                                                                                                                                                  | 103     | 150.00 |
|                                                                                                                                                                                                  | 104     | 700.00 |
|                                                                                                                                                                                                  | 105     | 450.00 |
|                                                                                                                                                                                                  | 106     | 200.00 |
| •                                                                                                                                                                                                | NULL    | NULL   |

## Nested Subqueries:

- **What:** A nested subquery is a subquery within another subquery, allowing for more complex logic and filtering.
- **When to Use:** When you need multiple layers of filtering or comparisons.
- **Example:** Find all customers who have placed orders with amounts greater than the average amount of all orders.

Unset

```
SELECT CustomerName
FROM Customers
WHERE CustomerID IN (
    SELECT DISTINCT CustomerID
    FROM Orders
    WHERE Amount > (SELECT AVG(Amount) FROM Orders)
);
```

This query counts the number of customers in the Customers table who have placed orders with an amount greater than 300. It uses a subquery to identify relevant CustomerIDs from the Orders table, returning a total count of those customers.

### Output:

| Result Grid |              |
|-------------|--------------|
|             | CustomerName |
| ▶           | Alice        |
|             | Charlie      |
|             | David        |

## Notes:

- **Single-row subqueries** return only one row and are used with single-value comparison operators.

- **Multi-Row Subquery:** This returns multiple customer names based on a condition from another table.
- **Multiple Column Subquery:** This compares two columns from the main query against two columns from a subquery.
- **Correlated Subquery:** This involves a subquery that references a column from the outer query, allowing it to be evaluated for each row processed by the outer query.
- **Nested Subquery:** This uses a subquery within another subquery to filter results based on aggregated data.

These types of subqueries allow for powerful data retrieval and manipulation, especially useful in e-commerce databases for querying customer behaviours, product data, and sales patterns.

# THANK YOU



**Stay updated with us!**



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)